# Sample Deparsing

```
control MyDeparser(packet_out      packet,
                   in my_headers_t hdr)
{
    apply {
        /* Layer 2 */
        packet.emit(hdr.ethernet);
        packet.emit(hdr.vlan_tag);

        /* Layer 2.5 */
        packet.emit(hdr.mpls);

        /* Layer 3 */
           /* ARP */
        packet.emit(hdr.arp);
        packet.emit(hdr.arp_ipv4);
           /* IPv4 */
        packet.emit(hdr.ipv4);
           /* IPv6 */
        packet.emit(hdr.ipv6);

        /* Layer 4 */
        packet.emit(hdr.icmp);
        packet.emit(hdr.tcp);
        packet.emit(hdr.udp);
    }
}
```

- **Assembling the packet from headers**
- **Expressed as another control function**
  - No need for another construct
- **packet_out – defined in core.p4**
  - emit(header) – serialize the header **if** it is valid
  - emit(header_stack) – serialize the valid elements in order
- **Advantages over p4_14:**
  - Decoupling of parsing and deparsing

# Agenda

- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary

# Overall P4 Program Structure

```
/* -*- P4_16 -*- */
#include <core.p4>
#include <v1model.p4>

/*********************  T Y P E S  ***********************/
typedef bit<48>   mac_addr_t;
header  ethernet_t   { /* Slide 30 */ }
struct  my_headers_t { /* Slide 30 */ }

/*****************  C O N S T A N T S  *******************/
const mac_addr_t BROADCAST_MAC = 0xFFFFFFFFFFFF;

/***********  P A R S E R S and C O N T R O L S  ***********/
parser  MyParser(...)                { /* Slide 38 */ }
control MyVerifyChecksum(...)        { /* Slide 76 */ }
control MyIngress(...)               { /* Slide 44 */ }
control MyEgress(...)                { . . . . }
control MyComputeChecksum(...)       { /* Slide 76 */ }
control MyDeparser(...)              { /* Slide 65 */ }

/***************  F U L L   P A C K A G E  ***************/
V1Switch(
    MyParser(), MyVerifyChecksum(), MyIngress(),
    MyEgress(), MyComputeChecksum(), MyDeparser()
) main;
```

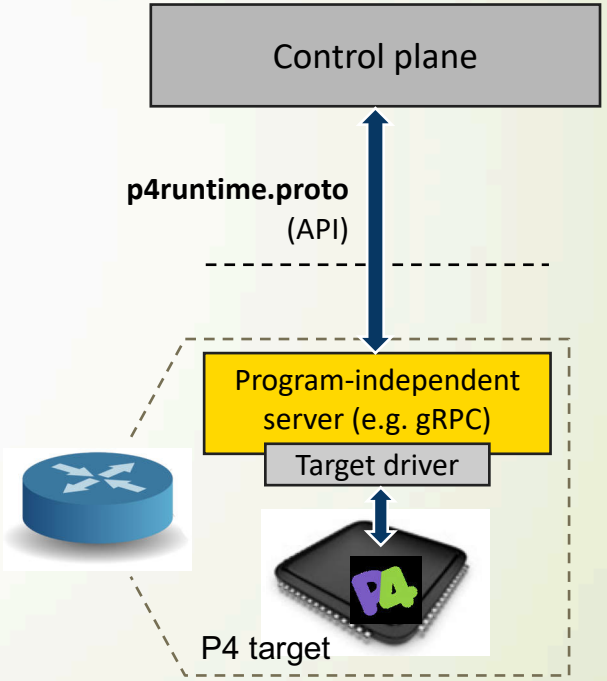
- **Start with Emacs-style comment to select the proper editor mode**
- **Include the core library**
- **Include the architecture-specific file(s)**
- **Define Types**
  - typedefs, headers, structs, ...
- **Define Constants**
- **Define Parsers and Controls**
- **Assemble the top-level controls in a package**
  - Package is defined by the Architecture
    - Represents the set of programmable P4 components and their interfaces
  - The name of the package must be **`main`**
- **That's it!**

# Agenda

- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary

# What is P4Runtime?

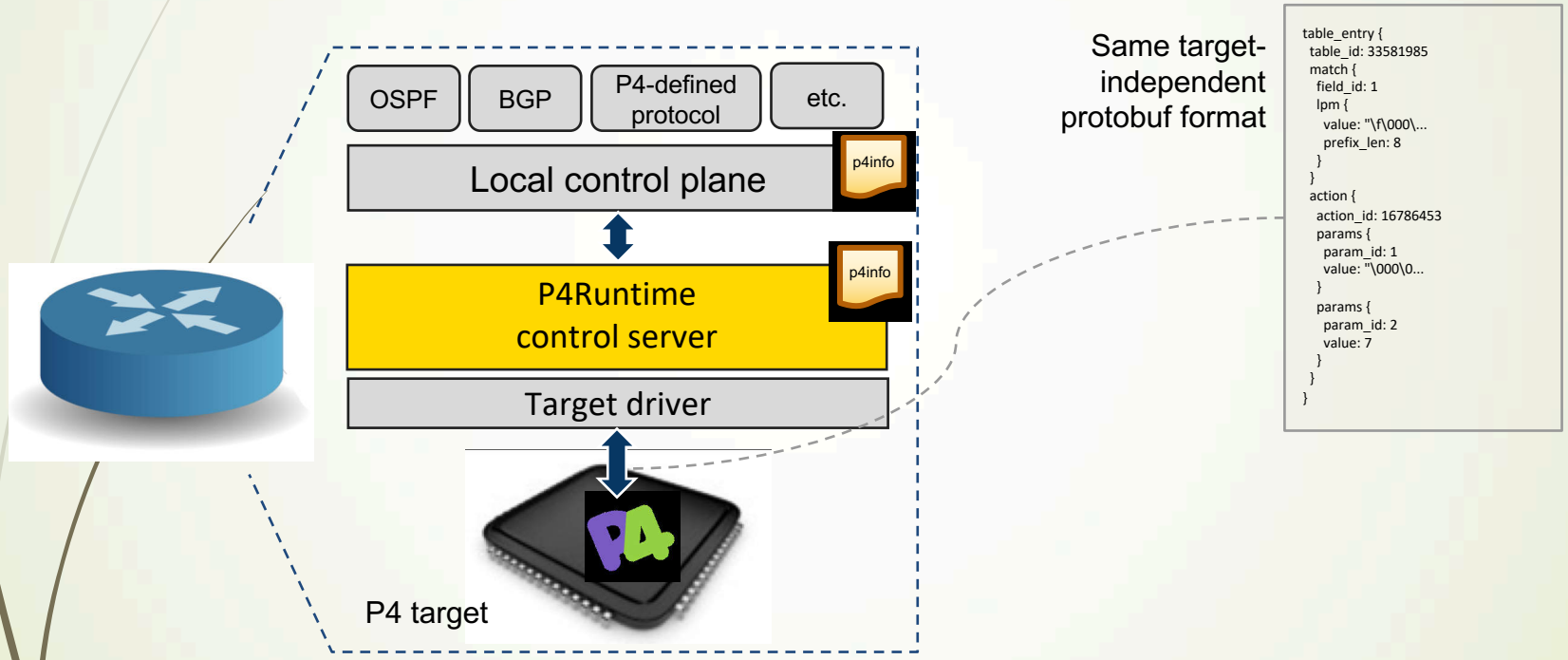
- **Framework for runtime control of P4 targets**
  - Open-source API + server implementation
    - https://github.com/p4lang/PI
  - Initial contribution by Google and Barefoot
- **Work-in-progress by the p4.org API WG**
  - Draft of version 1.0 available
- **Protobuf-based API definition**
  - p4runtime.proto
  - gRPC transport
- **P4 program-independent**
  - API doesn't change with the P4 program
- **Enables field-reconfigurability**
  - Ability to push new P4 program without recompiling the software stack of target switches



# Properties of Runtime Control APIs

| API | Target-independent | Protocol-independent |
|---|---|---|
| P4 compiler auto-generated | | ✗ |
| BMv2 CLI | ✗ | |
| OpenFlow | | ✗ |
| SAI | | ✗ |
| P4Runtime | | |

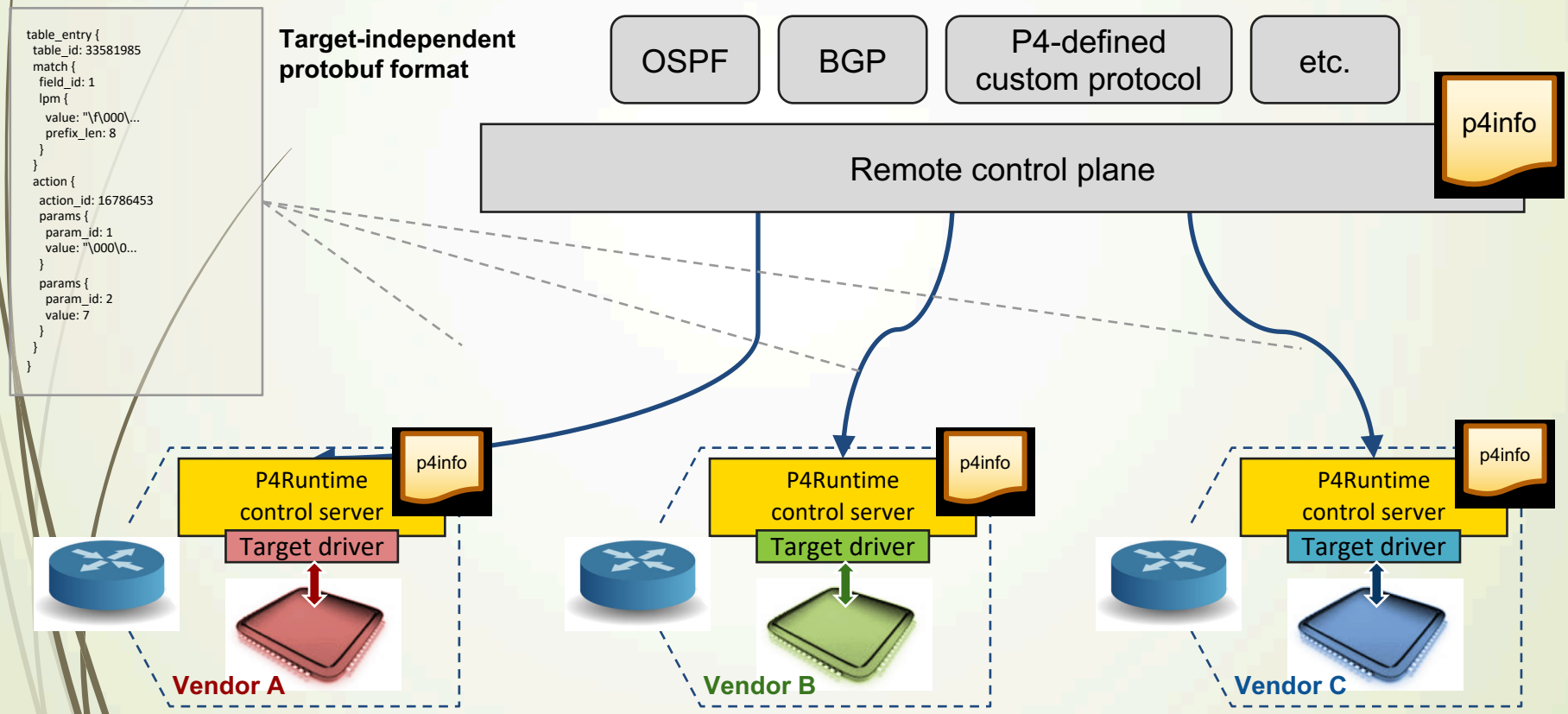
# Local Control Plane




P4Runtime can be used equally well by a remote or local control plane

# Remote Control Plane

Sample Controller: ONOS



# Agenda

- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary

# Advantages of $P4_{16}$

- **Clearly defined semantics**
  - You can describe what your data plane program is doing
- **Expressive**
  - Supports a wide range of architectures through standard methodology
- **High-level, Target-independent**
  - Uses conventional constructs
  - Compiler manages the resources and deals with the hardware
- **Type-safe**
  - Enforces good software design practices and eliminates “stupid” bugs
- **Agility**
  - High-speed networking devices become as flexible as any software
- **Insight**
  - Freely mixing packet headers and intermediate results

**We couldn’t cover everything, please see tutorials and presentations at p4.org**

# Questions

chsu@cs.nthu.edu.tw